

Home Lab Project Week 5, Day 3

To analyze captured FTP and HTTP traffic in Wireshark and extract unencrypted authentication details, use the following step-by-step workflow:

1. Filter the Traffic

Open your capture file in Wireshark and use the display filter bar at the top to isolate the target protocol:

- **For FTP:** Type `ftp` or `ftp-data` and press Enter.
- **For HTTP:** Type `http` and press Enter.

2. Isolate the Session (Follow TCP Stream)

Following the TCP stream reconstructs the disjointed packets into a chronological, readable transcript of the entire session:

1. **Right-click** on any packet belonging to the conversation you want to inspect (e.g., an `HTTP POST` request or an `FTP USER` command).
2. Hover over **Follow** in the context menu.
3. Select **TCP Stream**.

3. Identify Cleartext Credentials

A new window will open displaying the full text exchange. **Red text** typically indicates data sent from the client, while **blue text** represents responses from the server.

Locating Credentials in FTP Streams

FTP transmits commands and text responses using plain ASCII. In the followed stream, look for the standard [FTP authentication command sequence](#):

- **Username:** Look for the prefix `USER` followed immediately by the plain text account name (e.g., `USER anonymous` or `USER john_doe`).
- **Password:** Look directly below the username request for the prefix `PASS` followed by the unencrypted password (e.g., `PASS Secret123`).

Locating Credentials in HTTP Streams

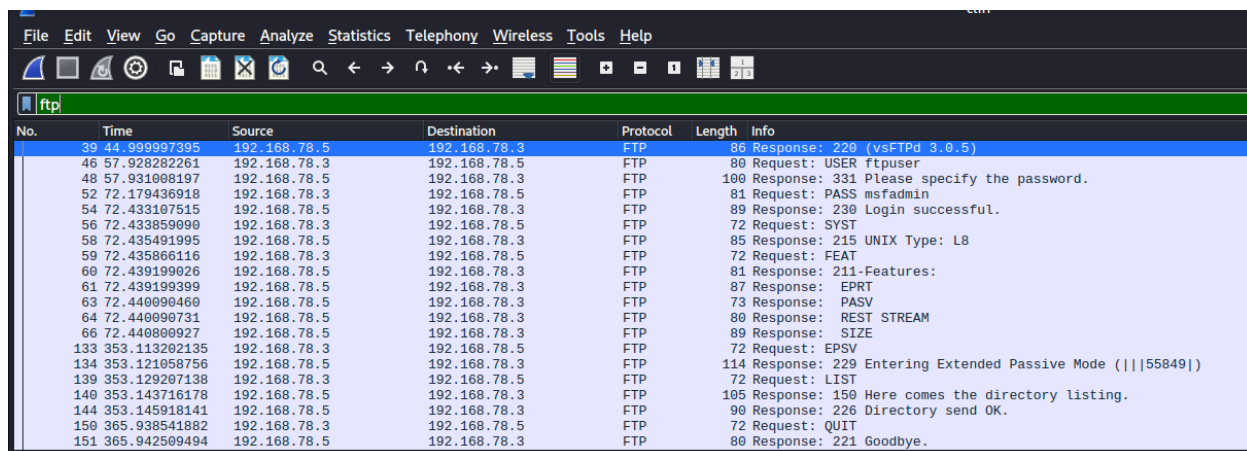
Unencrypted HTTP credentials usually reside inside the request body of `POST` methods sent during a login attempt:

- **HTML Form Submissions:** Look for a string at the bottom of the first client header block containing fields like `username=admin&password=password125`.
- **Basic Authentication:** Look closely at the HTTP headers for a line starting with `Authorization: Basic` . While this looks scrambled, it is actually encoded in **Base64** (not encrypted). You can copy that string into any Base64 decoder to instantly reveal the `username:password` format.

FTP Traffic Analysis

Now in Wireshark we can generate the FTP traffic and capture it for **analysis**.

To filter FTP traffic we type `ftp`.



No.	Time	Source	Destination	Protocol	Length	Info
39	44.999997395	192.168.78.5	192.168.78.3	FTP	86	Response: 220 (vsFTPd 3.0.5)
46	57.928282261	192.168.78.3	192.168.78.5	FTP	80	Request: USER ftpuser
48	57.931098197	192.168.78.5	192.168.78.3	FTP	100	Response: 331 Please specify the password.
52	72.179436918	192.168.78.3	192.168.78.5	FTP	81	Request: PASS msfadmin
54	72.433107515	192.168.78.5	192.168.78.3	FTP	89	Response: 230 Login successful.
56	72.433859090	192.168.78.3	192.168.78.5	FTP	72	Request: SYST
58	72.435491995	192.168.78.5	192.168.78.3	FTP	85	Response: 215 UNIX Type: L8
59	72.435866116	192.168.78.3	192.168.78.5	FTP	72	Request: FEAT
60	72.439199026	192.168.78.5	192.168.78.3	FTP	81	Response: 211-Features:
61	72.439199399	192.168.78.5	192.168.78.3	FTP	87	Response: EPRT
63	72.440090460	192.168.78.5	192.168.78.3	FTP	73	Response: PASV
64	72.440090731	192.168.78.5	192.168.78.3	FTP	80	Response: REST STREAM
66	72.440809927	192.168.78.5	192.168.78.3	FTP	89	Response: SIZE
133	353.113202135	192.168.78.3	192.168.78.5	FTP	72	Request: EPSV
134	353.121058756	192.168.78.5	192.168.78.3	FTP	114	Response: 229 Entering Extended Passive Mode (55849)
139	353.129207138	192.168.78.3	192.168.78.5	FTP	72	Request: LIST
140	353.143716178	192.168.78.5	192.168.78.3	FTP	105	Response: 150 Here comes the directory listing.
144	353.145918141	192.168.78.5	192.168.78.3	FTP	90	Response: 226 Directory send OK.
150	365.938541882	192.168.78.3	192.168.78.5	FTP	72	Request: QUIT
151	365.942509494	192.168.78.5	192.168.78.3	FTP	80	Response: 221 Goodbye.

The password is in plain text

Length	Info
86	Response: 220 (vsFTPd 3.0.5)
80	Request: USER ftpuser
100	Response: 331 Please specify the password.
81	Request: PASS msfadmin
89	Response: 230 Login successful.
72	Request: SYST

HTTP Traffic Analysis

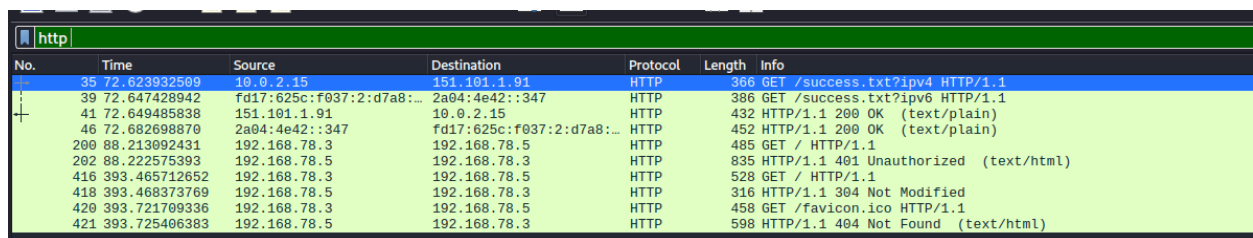
Step 5 — Find the login in Wireshark

Return to Wireshark while the capture is running.

5.1 Find the authentication request

Use this display filter:

http.authorization or just http



No.	Time	Source	Destination	Protocol	Length	Info
35	72.623932509	10.0.2.15	151.101.1.91	HTTP	366	GET /success.txt?ipv4 HTTP/1.1
39	72.647428942	fd17:625c:f037:2:d7a8::...	2a04:4e42::347	HTTP	386	GET /success.txt?ipv6 HTTP/1.1
41	72.649485838	151.101.1.91	10.0.2.15	HTTP	432	HTTP/1.1 200 OK (text/plain)
46	72.682698870	2a04:4e42::347	fd17:625c:f037:2:d7a8::...	HTTP	452	HTTP/1.1 200 OK (text/plain)
200	88.213092431	192.168.78.3	192.168.78.5	HTTP	485	GET / HTTP/1.1
202	88.222575393	192.168.78.5	192.168.78.3	HTTP	835	HTTP/1.1 401 Unauthorized (text/html)
416	393.465712652	192.168.78.3	192.168.78.5	HTTP	528	GET / HTTP/1.1
418	393.468373769	192.168.78.5	192.168.78.3	HTTP	316	HTTP/1.1 304 Not Modified
420	393.721709336	192.168.78.3	192.168.78.5	HTTP	458	GET /favicon.ico HTTP/1.1
421	393.725406383	192.168.78.5	192.168.78.3	HTTP	598	HTTP/1.1 404 Not Found (text/html)

We need to look for the HTTP request to Ubuntu, then select it and expand the Hypertext Transfer Protocol section in the packet details.

Find the header named:

Authorization: Basic ...

That is the HTTP Basic Authentication header.

5.2 Follow the TCP stream

Right-click the HTTP request and select Follow → TCP Stream.

You should see a request containing a header similar to:

GET / HTTP/1.1

Host: 192.168.5

Authorization: Basic [encoded credentials]

The actual header will contain a Base64-encoded value.

5.3 Understand what you captured

HTTP Basic Authentication encodes the username and password together using Base64. Base64 is not encryption. Anyone who can capture the unencrypted HTTP traffic can decode the credentials.

9054	998.382116764	192.168.78.3	192.168.78.5	HTTP	415 GET /favicon.ico HTTP/1.1
9439	1452.064122809	192.168.78.3	192.168.78.5	HTTP	187 GET / HTTP/1.1
9050	998.256219692	192.168.78.3	192.168.78.5	HTTP	400 GET / HTTP/1.1

But we did not have to decode it because it's displayed in the packet details and authorization in Hypertext Transfer Protocol

```
▶ Frame 21: Packet, 528 bytes on wire (4224 bits), 528 bytes captured (4224 bits) on interface any, id 0
▶ Linux cooked capture v1
▶ Internet Protocol Version 4, Src: 192.168.78.3, Dst: 192.168.78.5
▶ Transmission Control Protocol, Src Port: 55528, Dst Port: 80, Seq: 1, Ack: 1, Len: 460
▼ Hypertext Transfer Protocol
  ▶ GET / HTTP/1.1\r\n
  Host: 192.168.78.5\r\n
  User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0\r\n
  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8\r\n
  Accept-Language: en-US,en;q=0.5\r\n
  Accept-Encoding: gzip, deflate\r\n
  ▼ Authorization: Basic bGFidXNlcjpsYWJwYXNz\r\n
    Credentials: labuser:labpass
  Connection: keep-alive\r\n
  Upgrade-Insecure-Requests: 1\r\n
  If-Modified-Since: Fri, 25 Sep 2026 20:00:46 GMT\r\n
  If-None-Match: "44-65c542c14025e"\r\n
  Priority: u=0, i\r\n
  \r\n
  [Response in frame: 23]
  [Full request URI: http://192.168.78.5/]
```

Step 6 — Documenting the results

Home lab report, record:

- Objective: Capture a simulated HTTP login from Kali to Ubuntu.
- Server: Ubuntu running Apache with HTTP Basic Authentication.
- Client: Kali Linux running Firefox and Wireshark.
- Protocol: HTTP over TCP port 80.
- Evidence: HTTP authorization header captured in Wireshark.
- Finding: The Basic Authentication credentials were recoverable because HTTP does not encrypt the traffic.
- Security recommendation: Use HTTPS with TLS to protect credentials in transit.

Important cleanup

Now that we finished, we need to stop the capture and remove the lab login configuration if we no longer need it. On Ubuntu let's type in:

```
sudo a2disconf lab-login
sudo systemctl reload apache2
```

```
Maiboxuser@Ubuntu:~$ sudo a2disconf lab-login
[sudo: authenticate] Password:
Conf lab-login disabled.
To activate the new configuration, you need to run:
systemctl reload apache2
boxuser@Ubuntu:~$ sudo systemctl reload apache2
boxuser@Ubuntu:~$
```

This restores access to the test page without that authentication requirement. Keep the lab password out of screenshots and reports.

Your next checkpoint: After logging in from Firefox, check whether Wireshark shows an **Authorization: Basic** header. If it does, you've successfully captured the simulated login.